

InboxLift

InboxLift is an **email warming** platform: it connects real mailboxes over SMTP/IMAP, exchanges realistic-looking messages between your accounts on a controlled schedule, and tracks whether messages land in the inbox or spam. The goal is to improve **sender reputation** and reduce false positives from spam filters when you later send real campaigns or transactional mail from those addresses.

Why email “warnings” and deliverability matter

Modern inbox providers (Google Workspace, Microsoft 365, Yahoo, etc.) do not treat every new sender equally. A domain or IP with **no history** of legitimate, engaged traffic is more likely to see:

- Messages sorted into **Spam** or **Promotions**, even when content is benign
- **Throttling** or temporary blocks after sudden volume spikes
- **Bounce and complaint signals** that permanently hurt reputation if ignored

Filters use signals such as **sending volume patterns**, **engagement** (opens, replies, moves out of spam), **authentication** (SPF, DKIM, DMARC), **list quality**, and **consistency over time**. “Warming” does not replace authentication or good list hygiene, but it can help **establish a baseline** of normal, two-way mail flow so future sends are less likely to trigger aggressive filtering.

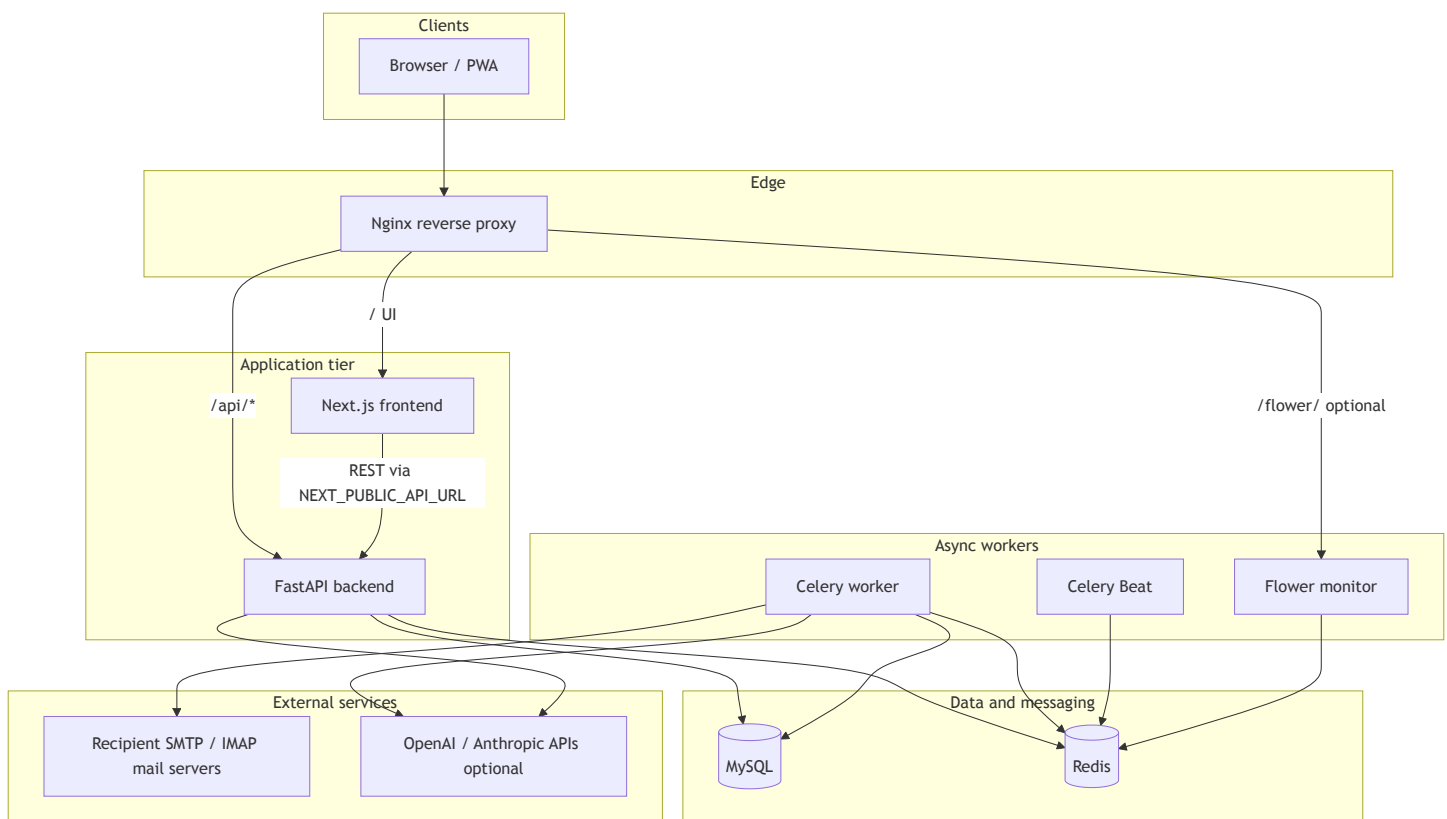
Important: Use InboxLift only with mailboxes you own or are authorized to automate. Respect your provider’s terms of service and applicable anti-spam laws. Warming is a **reputation tool**, not a way to bypass filters for unsolicited mail.

What InboxLift does

- **Account linking:** Store SMTP/IMAP settings and encrypted credentials per user.
- **Scheduled warming:** Celery Beat runs warming cycles (default: every 30 minutes) and pairs active accounts to send warm-up messages.

- **AI-assisted copy:** Optional OpenAI or Anthropic integration generates subject/body text; otherwise template-based content is used.
- **Inbox checks:** Periodic IMAP checks classify delivery (e.g. inbox vs. spam) and support reply flows aligned with configured reply rates.
- **Ramp-up:** Per-account daily limits increase over a configurable ramp (see `WARMUP_*` settings in the backend config).
- **Dashboard:** Next.js frontend for managing accounts and viewing analytics.

Architecture



Flow summary:

1. Users interact with the **Next.js** app; the app calls the **FastAPI** API (`/api/v1/...`).
2. **MySQL** holds users, email accounts (encrypted secrets), warm-up message records, and analytics snapshots.
3. **Redis** backs Celery's broker and result backend; separate logical DB indexes are used for app cache vs. Celery.
4. **Celery worker** sends mail via SMTP and checks placement via IMAP; optional **AI** calls generate message text.
5. **Celery Beat** schedules warming, delivery checks, daily analytics, and counter resets.

6. **Nginx** (optional in Docker Compose) terminates HTTP, rate-limits auth/API routes, and can proxy **Flower** with HTTP basic auth.

Prerequisites

- **Docker** and **Docker Compose** (recommended), *or*
- **Python 3.12+**, **Node.js 22+**, **MySQL 8+**, and **Redis 7+** for local development.

For AI-generated warm-up content, an **OpenAI** and/or **Anthropic** API key is optional (the app falls back to templates if keys are missing).

Installation (Docker Compose)

From the repository root:

1. **Clone the repository** (if you have not already).
2. **Create an environment file** (optional but recommended). Compose reads variables from your shell or a `.env` file in the same directory as `docker-compose.yml`. Example:

```
# Database (MySQL application user and root; defaults match docker-compose)
MYSQL_PASSWORD=your_secure_password
MYSQL_ROOT_PASSWORD=your_secure_root_password

# Security – use long random values in production
SECRET_KEY=your-jwt-signing-secret-at-least-32-chars
ENCRYPTION_KEY=exactly-32-char-fernet-key!!

# Optional AI
OPENAI_API_KEY=sk-...
ANTHROPIC_API_KEY=sk-ant-...

# Frontend → backend URL (browser must reach this host)
NEXT_PUBLIC_API_URL=http://localhost:8000

ENVIRONMENT=development
DEBUG=true
```

ENCRYPTION_KEY must be suitable for Fernet-style encryption (the default dev string in code is 32 characters; **replace it in production**).

3. Build and start all services:

```
docker compose up --build
```

On first start, the backend container runs **alembic upgrade head** then **Uvicorn** with reload. MySQL and Redis must pass health checks before the API starts.

4. Access the stack:

Service	URL / port
API	http://localhost:8000
API docs (dev)	http://localhost:8000/api/docs
Frontend	http://localhost:3000
Nginx (Compose)	http://localhost:80
Flower	http://localhost:5555 (direct); via Nginx: <code>/flower/</code> (requires <code>auth_basic</code> — see below)
MySQL	localhost:3306
Redis	localhost:6379

5. **Flower behind Nginx:** `nginx/nginx.conf` references `/etc/nginx/.htpasswd` for `/flower/`. The default `docker-compose.yml` does not mount that file, so either add a volume for a generated `.htpasswd` into the Nginx container or use Flower directly at <http://localhost:5555> during development.

Local development (without full Compose)

Run infrastructure only, then the apps on the host:

```
docker compose up mysql redis -d
```

Backend

```
cd backend
python -m venv .venv
source .venv/bin/activate # Windows: .venv\Scripts\activate
pip install -r requirements.txt
```

Set `DATABASE_URL` , `REDIS_URL` , `CELERY_BROKER_URL` , `CELERY_RESULT_BACKEND` , `SECRET_KEY` , and `ENCRYPTION_KEY` in `backend/.env` (see `app/core/config.py` for defaults and names).

```
alembic upgrade head
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

In separate terminals (same venv and env):

```
celery -A app.tasks.celery_app.celery_app worker --loglevel=info
celery -A app.tasks.celery_app.celery_app beat --loglevel=info
```

Frontend

```
cd frontend
npm ci --legacy-peer-deps
export NEXT_PUBLIC_API_URL=http://localhost:8000
npm run dev
```

Open <http://localhost:3000>.

Configuration reference

Key settings are defined in `backend/app/core/config.py` and can be overridden via environment variables:

Variable	Purpose
<code>DATABASE_URL</code>	Async SQLAlchemy URL (<code>mysql+aiomysql://user:pass@host:3306/dbname</code>)

Variable	Purpose
REDIS_URL	Application Redis
CELERY_BROKER_URL / CELERY_RESULT_BACKEND	Celery broker and results
SECRET_KEY	JWT signing
ENCRYPTION_KEY	Encrypts stored mailbox passwords
CORS_ORIGINS	JSON array of allowed browser origins
DEFAULT_WARMUP_DAILY_LIMIT , WARMUP_RAMP_UP_DAYS , WARMUP_START_EMAILS_PER_DAY , WARMUP_MAX_EMAILS_PER_DAY , WARMUP_REPLY_RATE	Warming schedule and behavior
OPENAI_API_KEY / ANTHROPIC_API_KEY	Optional AI providers
NEXT_PUBLIC_API_URL	Browser-visible API base URL (frontend)

Scheduled tasks are registered in `backend/app/tasks/celery_app.py` (warm-up cycle every 30 minutes, delivery checks hourly, analytics and daily resets at fixed UTC times).

Testing

Backend (from `backend/`):

```
pytest
```

Frontend:

```
cd frontend && npm test
```

Project layout

- backend/ — FastAPI app, SQLAlchemy models, Alembic migrations, Celery tasks
- frontend/ — Next.js 15 UI
- nginx/ — Reverse proxy configuration for the Compose stack

License and compliance

Use InboxLift in compliance with your email provider's policies and regulations such as CAN-SPAM, GDPR (where applicable), and your organization's acceptable use policy. The authors are not responsible for misuse.